



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Instituto Tecnológico de Tepic

Ingeniería en sistemas computacionales

Programación Web

Actividad 1: Programación del lado del cliente vs del lado del servidor

Facilitador: Nava Hernández Irving Yair

Alumna: Cervantes Pérez Valeria

Número de control: 23400522

Séptimo semestre

Tepic, Nayarit, 20 de julio de 2026

Programación del lado del servidor

La programación del lado del servidor (o desarrollo backend) engloba la lógica, la gestión de datos y los procesos informáticos que se ejecutan directamente en un servidor web antes de enviar la respuesta final al navegador del usuario.

Conceptos clave de la programación del servidor

1. Servidor web y lógica de negocio

Un servidor web es la computadora (física o virtual) que recibe las peticiones HTTP de los clientes (navegadores o aplicaciones móviles). Su función principal es procesar la lógica de negocio, es decir, aplicar las reglas del sistema (como verificar credenciales, calcular subtotales o filtrar catálogos) antes de generar una respuesta.

2. Generación de contenido dinámico (Server-Side Rendering)

A diferencia de las páginas estáticas cuyo código no cambia, el servidor utiliza plantillas (templates) junto con datos en tiempo real para construir dinámicamente archivos HTML, CSS o JSON que se envían al cliente. De este modo, un solo diseño de plantilla puede mostrar información personalizada para millones de usuarios.

3. Interacción con Bases de Datos y Almacenamiento

El código del lado del servidor se encarga de consultar, insertar, actualizar o eliminar información en sistemas de bases de datos (como MySQL, PostgreSQL, Oracle, MongoDB). Esto permite gestionar cuentas de usuario, carritos de compras o inventarios de forma estructurada.

4. Interfaces de Programación de Aplicaciones (APIs)

Una API en el lado del servidor expone un conjunto de rutas o puntos de acceso (endpoints) a los que otras aplicaciones o clientes (como apps móviles o frontends modernos) pueden enviar solicitudes para consumir o enviar datos, generalmente formateados en JSON o XML.

5. Autenticación, Autorización y Seguridad

Debido a que el código del servidor no está visible para el usuario final, es el lugar ideal para implementar la seguridad del sistema:

- Autenticación y permisos: Control de inicio de sesión y verificación de lo que cada usuario puede ver o modificar.
- Validación de entradas: Sanitización de datos para evitar vulnerabilidades comunes como la inyección SQL o XSS.

6. Rendimiento, Almacenamiento en Caché y Serverless

- Almacenamiento en caché del servidor (Server-Side Caching): Guarda en memoria resultados o páginas previamente generadas para responder más rápido a solicitudes posteriores sin sobrecargar la base de datos.
- Arquitectura Serverless: Un modelo donde la ejecución de código se abstrae de la gestión del servidor físico; el proveedor en la nube escala y ejecuta los scripts dinámicamente según la demanda.

Lenguajes y Frameworks principales

Los lenguajes del lado del servidor proporcionan bibliotecas y estructuras (frameworks) para abstraer tareas de bajo nivel (como el manejo del protocolo HTTP o conexiones a bases de datos):

Lenguaje	Frameworks representativos	Características destacadas
PHP	Laravel	Históricamente el más usado para la web dinámicas; fácil conexión a bases de datos como MySQL.
Python	Django	Sintaxis clara y humana; enfocado en desarrollo rápido y alta seguridad.
JavaScript	Express (Node.js)	Permite usar JavaScript tanto en <i>frontend</i> como en <i>backend</i> ; ideal para procesos asíncronos o en tiempo real.
C# / .NET	ASP.NET	Ecosistema comercial de Microsoft; división clara

		entre capas de diseño y código.
Ruby	Ruby on Rails (Rails)	Enfocado en la simplicidad y rapidez de desarrollo mediante el uso de convenciones.

Programación del lado del cliente

La programación del lado del cliente (o desarrollo frontend) abarca toda la lógica, la presentación visual y la interactividad que se ejecutan directamente en el navegador web o dispositivo del usuario final.

Conceptos clave de la programación del cliente

1. El modelo cliente-servidor y el dispositivo final

En la arquitectura web, los dispositivos de los usuarios (como computadoras o teléfonos inteligentes) actúan como clientes. El scripting del lado del cliente se refiere al código descargado desde el servidor que el motor del navegador web (como V8 en Chrome o Gecko en Firefox) interpreta y ejecuta de forma local.

2. Trinomio fundamental del Frontend: HTML, CSS y JavaScript

La separación de responsabilidades (Separation of Concerns) es el principio fundamental para construir la interfaz:

- **HTML (Estructura):** Define la jerarquía, los bloques de construcción y el contenido de la página (texto, imágenes, formularios).
- **CSS (Presentación y Estilo):** Modifica el aspecto visual, el diseño responsivo y la disposición de los elementos. Los preprocesadores CSS (como Sass o LESS) extienden sus capacidades agregando variables y reutilización de estilos.
- **JavaScript (Comportamiento):** Aporta la lógica dinámica, maneja eventos del usuario y modifica el contenido sin recargar la página.

3. El Modelo de Objetos del Documento (DOM)

El DOM (Document Object Model) es la representación estructurada en árbol del documento HTML en la memoria del navegador. Las aplicaciones cliente interactúan constantemente con el DOM mediante JavaScript para modificar elementos, actualizar texto o reaccionar a eventos (como clics de ratón o pulsaciones de teclas) en tiempo real.

4. Peticiones asíncronas y AJAX

Mediante AJAX (Asynchronous JavaScript and XML) o la API Fetch, el navegador puede realizar solicitudes en segundo plano al servidor para obtener o enviar datos (comúnmente en formato JSON) sin necesidad de refrescar toda la página web. Esto proporciona una experiencia de usuario fluida e interactiva.

5. Aplicaciones de Página Única (SPA) y Frameworks Modernos

A diferencia del enfoque imperativo clásico (manipular el DOM paso a paso, como se hacía tradicionalmente con bibliotecas como jQuery), los frameworks de SPA modernos utilizan un enfoque declarativo y enlace de datos (databinding):

- **Lógica declarativa:** Los cambios en los datos del modelo se reflejan automáticamente en la interfaz sin necesidad de escribir código manual para ocultar o mostrar elementos.
- **Componentización:** Se construye la aplicación mediante piezas independientes y reutilizables de interfaz de usuario.

Tecnologías y Herramientas Principales

Categoría	Tecnologías / Herramientas	Función y características
Estructura y Estilos	HTML5, CSS3, Sass, LESS	Creación de contenido, diseño responsivo y gestión de reglas de estilo reutilizables.
Lenguajes Fundamentales	JavaScript (ECMAScript)	Lenguaje dinámico estándar interpretado por todos los navegadores web.

Librerías Clásicas / DOM	jQuery	Simplifica la manipulación imperativa del DOM y peticiones AJAX.
Frameworks / Librerías SPA	React, Angular	Construcción de interfaces dinámicas basadas en componentes y enlace de datos declarativo.
Alternativas / WebAssembly	Blazor	Permite ejecutar código C# en el cliente en lugar de JavaScript mediante compilación web.

Cuadro comparativo: Cliente vs Servidor

Criterio	Programación del lado del cliente	Programación del lado del servidor
Descripción general	Gestiona la interfaz visual, el diseño, la presentación y las interacciones inmediatas con el usuario en el dispositivo final.	Maneja la lógica de negocio, el procesamiento dinámico de datos, el almacenamiento persistente y las reglas del sistema.
Lenguajes más utilizados	• JavaScript (estándar universal) • HTML5 (marcado) • CSS3 (estilo)	• PHP • Python • JavaScript (Node.js) • C# (.NET) • Ruby
Tecnologías y herramientas comunes	• <i>Frameworks/Librerías</i>: React, Angular, jQuery • <i>Preprocesadores CSS</i>: Sass, LESS • <i>Alternativas</i>: Blazor (C#)	• Frameworks: Django, Laravel, Express.js, ASP.NET Core, Ruby on Rails • Bases de Datos: MySQL, PostgreSQL, MongoDB, Oracle
Ventajas	• Respuestas e interacciones visuales inmediatas. • Reduce la carga sobre el servidor al procesar eventos localmente. • Permite interfaces enriquecidas y dinámicas mediante peticiones	• Permite generar contenido dinámico personalizado para millones de usuarios a partir de plantillas. • Acceso directo a bases de datos y archivos centralizados.

	asíncronas (AJAX / Fetch).	• Mayor capacidad para procesamiento intensivo de datos.
Desventajas	• Depende del rendimiento, hardware y tipo de navegador del dispositivo del usuario. • El código fuente es público y expuesto en el navegador. • Incompatibilidades entre versiones de navegadores.	• Genera latencia por los tiempos de red (cada solicitud debe viajar cliente-servidor). • Consume recursos constantes del servidor, requiriendo escalabilidad ante alto tráfico.
Seguridad	• Menos seguro para lógica sensible: El código se descarga en el cliente y puede ser inspeccionado o manipulado. • Riesgos: Ataques de Scripting entre Sitios (XSS).	• Más seguro para datos sensibles: El código se ejecuta de forma oculta en el servidor. • Encargado de la autenticación, control de permisos y sanitización contra inyección SQL.
Ubicación de ejecución	Navegador web (u otra aplicación cliente en la computadora o dispositivo móvil del usuario).	Servidor web (físico, virtual o en la nube) que procesa la solicitud antes de responder.

Referencias bibliográficas:

- Axarnet. (s. f.). Lenguajes del lado del servidor: cuáles son los más utilizados y qué características tienen. Blog de Axarnet.
<https://axarnet.es/blog/lenguajes-del-lado-del-servidor>
- Lenovo México. (s. f.). ¿Qué es la programación en el lado del servidor? Glosario Lenovo. <https://www.lenovo.com/mx/es/glosario/programacion-del-servidor/>
- MDN Web Docs. (2025, 27 de marzo). Programación lado servidor. Mozilla Developer Network.
https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side

- Cloudflare. (s. f.). ¿Qué significa lado del cliente y lado del servidor? | Lado del cliente vs. Lado del servidor. Centro de Aprendizaje de Cloudflare. <https://www.cloudflare.com/es-es/learning/serverless/glossary/client-side-vs-server-side/>
- IBM. (2021, 9 de marzo). Programación. Documentación de IBM (IBM Docs). <https://www.ibm.com/docs/es/i/7.4.0?topic=serving-programming>
- Microsoft Learn. (2023, 10 de octubre). Tecnologías web comunes del lado del cliente. Guías de arquitectura de .NET. <https://learn.microsoft.com/es-es/dotnet/architecture/modern-web-apps-azure/common-client-side-web-technologies>